

Vectorise five loops, time both, draw one curve

You reproduce the speed-up table yourself, on your own machine, with your own data. The point is not the code — it is watching the numbers appear rather than being told them.

Guided lab

16 minutes

Not graded

Bring the ratio to Friday

[The harness](#)[The five jobs](#)[Shape check](#)[One curve](#)[If you finish early](#)[Reflection](#)

SETUP · 3 MINUTES

The harness

Open the provided `lab04.ipynb` and run cell 1. It is given to you — the lab is what comes after.

1

lab04.ipynb — cell 1, already in the notebook

PYTHON

```
import numpy as np, time
import matplotlib.pyplot as plt

N = 1_000_000
rng = np.random.default_rng(0)      # seed 0 — everyone gets identical data
xs = rng.random(N)
ys = rng.random(N)
xs_list, ys_list = list(xs), list(ys)

def timeit(f, reps=3):
    best = float("inf")
    for _ in range(reps):
        t0 = time.perf_counter()
        f()
        best = min(best, time.perf_counter() - t0)
    return best * 1000                # milliseconds
```



Why the best of three, and not the average

Your laptop is doing other things. A slow run tells you about the operating system; the fastest run is the closest you get to how long the work actually takes. Taking the minimum is the standard way to time small pieces of code, and it is why your three repeats matter.

TASK 1 · 8 MINUTES · THE MAIN TASK

The same five jobs, twice each

2

For each job: write the loop version, write the NumPy version, time both, and **check the two answers agree** before you believe either timing. A fast wrong answer is not a win.

- 1 **Square every element.** Loop: a comprehension over `xs_list`. NumPy: one operator on `xs`.
- 2 **Add the two lists together, element by element.** Loop: `zip` or an index. NumPy: one operator.
- 3 **Sum everything.** Loop: an accumulator, exactly as in Lab 1. NumPy: one method.
- 4 **Count how many values exceed 0.5.** Loop: a counter with an `if`. NumPy: try `(xs > 0.5).sum()` and work out for yourself why that gives a count.
- 5 **Find the largest value.** Loop: the "largest so far" pattern from Lab 1. NumPy: one method.

the shape of each answer – job 1 done for you

PYTHON

```
def loop_square():
    return [x * x for x in xs_list]

def numpy_square():
    return xs ** 2

print(timeit(loop_square), "ms    loop")
print(timeit(numpy_square), "ms    numpy")
print(np.allclose(loop_square(), numpy_square())) # must be True
```



Use `np.allclose`, not `=`

Floating-point arithmetic done in a different order gives answers that differ in the last decimal place, so exact equality can fail on two results that are both correct. `allclose` asks whether they agree to within a tiny tolerance, which is the question you actually mean. You will use it in every lab from here on.

Record your five ratios in a table. Then write the largest one at the top of your page — that is the number to bring on Friday.

On an ordinary laptop the NumPy versions come out somewhere between 30 and 200 times faster, depending on the job and the machine. Summing usually shows the largest gap; counting is often the smallest because the comparison builds an intermediate array.

If one of your ratios is close to 1, you have almost certainly timed the wrong thing — a common cause is calling `list()` on the NumPy result inside the timed function, which puts the loop back in.

TASK 2 · 2 MINUTES

Reproduce the silent bug on purpose

3

From part 2 of the lecture. Type it, predict, run, then fix it.

PYTHON

```
p = np.array([1., 2., 3.])
t = np.array([[1.], [2.], [3.]])

print(p.shape, t.shape)
print(((p - t) ** 2).mean())      # predictions are perfect. what prints?
print((p - t).shape)             # this line is the whole explanation
```

Then make the error go away by giving `t` the same shape as `p`. Two ways work — `t.reshape(3)` or `t.ravel()` — and both should give a mean squared error of exactly 0.0.

One curve, properly labelled

4

Draw the loss curve from part 4 of the lecture, and mark its minimum.

- 200 values of `w` from -2 to 4, and `loss = (w - 1.5)**2 + 0.5`.
- Plot loss against `w`, with both axes labelled and a title.
- Use `argmin` to find the best `w`, and add it as a single red point with `plt.scatter`.
- Add a legend that names the curve and the minimum.



Unlabelled axes are not finished

If your figure has no `xlabel` and `ylabel`, the task is not complete. This is the standard for every figure in this course from today, and it is worth marks in the midterm.

1.497... — not exactly 1.5, because `linspace` put 200 points on the interval and none of them landed precisely on the true minimum. That gap is worth noticing: your answer is only as fine as your grid, which is one of the reasons Friday's approach of *stepping downhill* beats today's approach of *trying two hundred values*.

If you finish early

- Rerun the timing with `N = 10_000_000`. Do the ratios stay the same, grow, or shrink? Have an explanation ready.
- Normalise `xs` so it has mean 0 and standard deviation 1 — one line, using broadcasting. Confirm with `xs_n.mean()` and `xs_n.std()`. You will do exactly this to real data in session 20.
- Plot three loss curves on one pair of axes, with minima at -1, 0.5 and 2, each labelled in the legend. This is the plot session 9 uses to explain the learning rate.
- The mistake I made while preparing the lecture.** My first rescaling version was `[(x - min(v)) / (max(v) - min(v)) for x in v]`. It looks fine. It never finished. Time it on *four thousand* elements — not a million — against a version with `min` and `max` hoisted out of the loop, and say in one sentence what the comprehension was hiding.

E **The loud broadcasting failure.** Task 2 was the *silent* bug; this is its noisy cousin. With `X = np.arange(12).reshape(3, 4)`, the line `X + np.array([10, 20, 30, 40])` works but `X + np.array([1, 2, 3])` raises an error. Explain why in terms of the two shapes, then repair it two *different* ways so that 1, 2 and 3 are added to the three rows.

Shapes are compared from the right: `(3, 4)` against `(3,)` puts 4 opposite 3, and neither is 1, so NumPy refuses. `(4,)` works because 4 matches 4.

Give the three numbers a column shape, `(3, 1)`, so they line up with the rows: `X + np.array([1, 2, 3]).reshape(3, 1)` or `X + np.array([1, 2, 3])[:, None]`. Both stretch the column across all four entries of each row.

Reflection



Write it in the notebook, in your own words

Explain broadcasting in plain English, without using the word "broadcast". Then say how a bias vector of shape `(3,)` could be added to every row of a batch of shape `(4, 3)` without writing a loop — you will do exactly that in session 8.